

14 — GenAI: Embeddings & RAG

Time: ~5-6 hours | **Level:** Advanced

What you'll learn:

- Text embeddings: how models represent meaning as vectors
- Embedding spaces: similarity, distance, and clustering
- Vector databases: storing and searching embeddings at scale
- Semantic search: finding relevant content by meaning, not keywords
- RAG (Retrieval-Augmented Generation): grounding LLMs with external knowledge
- Building a complete RAG pipeline for mental health clinical guidelines

Prerequisites: Notebooks 05-10 (NLP, Transformers, HuggingFace, fine-tuning)

The Biggest Gap in Most AI Engineers' Toolkit

Fine-tuning teaches a model *how to respond*. RAG teaches a model *what to respond with*.

Most production LLM applications use RAG — not fine-tuning alone — because:

- Knowledge can be **updated** without retraining
- Answers are **grounded** in real documents (reduces hallucination)
- You can **cite sources** for every answer

```
In [ ]: import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.decomposition import PCA

sns.set_theme(style='whitegrid', font_scale=1.1)
np.random.seed(42)
```

1. Text Embeddings — Meaning as Vectors

An **embedding** is a dense vector representation of text where:

- Similar meaning → similar vectors (close in space)
- Different meaning → different vectors (far apart)

Representation	Dimensions	Sparse?	Captures Meaning?
One-hot	vocab_size (50K+)	Yes	No
TF-IDF	vocab_size	Yes	Partially (word frequency)
Word2Vec	100-300	No	Word-level only
Sentence Embeddings	384-1024	No	Full sentence meaning

We'll use **sentence-transformers** — models specifically trained to produce meaningful sentence embeddings.

```
In [ ]: # — Sentence Embeddings with sentence-transformers —
from sentence_transformers import SentenceTransformer

# all-MiniLM-L6-v2: fast, 384-dim, great for semantic search
model = SentenceTransformer('all-MiniLM-L6-v2')

# Mental health clinical sentences
sentences = [
    # Depression-related
    "Patient reports persistent feelings of sadness and hopelessness.",
    "The client presents with low mood and loss of interest in activities.",
    "Depressive symptoms have worsened over the past three weeks.",
    # Anxiety-related
    "Patient experiences frequent panic attacks and excessive worry.",
    "Generalised anxiety disorder with somatic symptoms including chest tight",
    "The client reports severe social anxiety affecting daily functioning.",
    # Treatment-related
    "Cognitive behavioural therapy recommended as first-line treatment.",
    "SSRI medication initiated: sertraline 50mg daily.",
    # Unrelated
    "The weather forecast predicts rain tomorrow.",
    "Stock markets closed higher on Friday.",
]

embeddings = model.encode(sentences)

print(f'Number of sentences: {len(sentences)}')
print(f'Embedding dimension: {embeddings.shape[1]}')
print(f'Embedding shape: {embeddings.shape}')
print(f'\nFirst embedding (first 10 dims): {embeddings[0][:10].round(4)}')
print(f'\n💡 Each sentence is now a 384-dimensional vector.')
print(f'    Similar sentences → similar vectors → we can search by MEANING.')
```

2. Embedding Spaces & Similarity

Cosine similarity measures the angle between two vectors:

$$\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \cdot \|\mathbf{b}\|}$$

- **1.0** = identical meaning
- **0.0** = unrelated
- **-1.0** = opposite meaning (rare in practice)

Why cosine over Euclidean distance?

- Cosine is **magnitude-invariant** — only cares about direction
- Two sentences about depression will point in similar directions regardless of embedding norm

```
In [ ]: # — Similarity matrix between all sentences —————

sim_matrix = cosine_similarity(embeddings)

# Short labels for display
labels = [
    'Depression 1', 'Depression 2', 'Depression 3',
    'Anxiety 1', 'Anxiety 2', 'Anxiety 3',
    'Treatment CBT', 'Treatment SSRI',
    'Weather', 'Stocks'
]

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(sim_matrix, annot=True, fmt='.2f', cmap='RdYlBu_r',
            xticklabels=labels, yticklabels=labels,
            vmin=0, vmax=1, ax=ax)
ax.set_title('Cosine Similarity Between Clinical Sentences')
plt.tight_layout()
plt.show()

print('💡 Key observations:')
print(' • Depression sentences cluster together (high similarity ~0.6–0.8)')
print(' • Anxiety sentences cluster together')
print(' • Depression & anxiety have moderate similarity (both mental health)')
print(' • Weather/stocks are dissimilar to everything clinical (~0.0–0.1)')
```

```
In [ ]: # — 2D Visualisation of embedding space —————

pca = PCA(n_components=2)
embeddings_2d = pca.fit_transform(embeddings)

colors = ['#e74c3c'] * 3 + ['#3498db'] * 3 + ['#2ecc71'] * 2 + ['#95a5a6'] * 2
categories = ['Depression'] * 3 + ['Anxiety'] * 3 + ['Treatment'] * 2 + ['Unrelated'] * 2

fig, ax = plt.subplots(figsize=(10, 7))
for i, (x, y) in enumerate(embeddings_2d):
    ax.scatter(x, y, c=colors[i], s=100, zorder=5)
    ax.annotate(labels[i], (x, y), textcoords='offset points',
                xytext=(8, 8), fontsize=9)

# Add legend
for cat, col in [('Depression', '#e74c3c'), ('Anxiety', '#3498db'),
                ('Treatment', '#2ecc71'), ('Unrelated', '#95a5a6')]:
```

```

ax.scatter([], [], c=col, label=cat, s=100)
ax.legend(fontsize=10)

ax.set_title('Embedding Space (PCA 2D Projection)')
ax.set_xlabel(f'PC1 ({pca.explained_variance_ratio_[0]:.1%} variance)')
ax.set_ylabel(f'PC2 ({pca.explained_variance_ratio_[1]:.1%} variance)')
ax.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

print('💡 Related concepts cluster together in embedding space.')
print(' This is the foundation of semantic search and RAG.')

```

3. Vector Databases — Storing & Searching at Scale

Problem: Computing cosine similarity against 1M documents = 1M dot products per query. Too slow.

Solution: Vector databases use **Approximate Nearest Neighbour (ANN)** algorithms:

Algorithm		How It Works	Speed	Accuracy
HNSW		Graph-based — navigable small world	Very fast	Very high
IVF		Cluster-based — search nearest clusters only	Fast	High
LSH		Hash-based — similar vectors hash together	Fastest	Lower

Vector DB	Open Source	Managed	Key Feature
ChromaDB	✓	✗	Simplest to get started, Python-native
FAISS	✓	✗	Facebook's library, best for pure speed
Pinecone	✗	✓	Fully managed, scales automatically
Weaviate	✓	✓	Hybrid search (vector + keyword) built-in
Qdrant	✓	✓	Rich filtering, good for production

We'll use **ChromaDB** for hands-on practice.

```

In [ ]: # — ChromaDB: Create a vector store —————
import chromadb

# Create an in-memory ChromaDB client
client = chromadb.Client()

# Create a collection (like a table in a database)
collection = client.create_collection(
    name='mental_health_guidelines',
    metadata={'hnsw:space': 'cosine'} # use cosine similarity
)

```

```
# Mental health clinical guidelines (our knowledge base)
guidelines = [
    "Depression screening should use the PHQ-9 questionnaire. A score of 10",
    "First-line treatment for moderate depression includes CBT (Cognitive Be",
    "For treatment-resistant depression, consider augmentation with lithium,",
    "Generalised Anxiety Disorder (GAD) is diagnosed when excessive worry pe",
    "First-line treatment for GAD includes CBT and/or SSRI/SNRI medication.",
    "Panic disorder treatment involves CBT focusing on interoceptive exposur",
    "Suicidal ideation requires immediate risk assessment using the Columbia",
    "PTSD treatment guidelines recommend trauma-focused CBT or EMDR as first",
    "Bipolar disorder requires mood stabilisers such as lithium or valproate",
    "Sleep hygiene education should be provided to all patients with insomni",
    "Regular exercise (150 minutes per week) has demonstrated efficacy compa",
    "Patients on SSRI medication should be monitored for suicidal ideation c
]

# Add documents to the collection
collection.add(
    documents=guidelines,
    ids=[f'guideline_{i}' for i in range(len(guidelines))],
    metadatas=[{'topic': 'depression'} if 'depress' in g.lower()
                else {'topic': 'anxiety'} if 'anxi' in g.lower()
                else {'topic': 'general'} for g in guidelines]
)

print(f'Added {collection.count()} guidelines to ChromaDB')
print(f'Collection: {collection.name}')
print(f'\n💡 ChromaDB automatically embeds documents using its default model')
print(f'    In production, you\'d use your own embedding model for consistenc
```

4. Semantic Search Pipeline

Now we can **search by meaning**, not just keywords:

```
Query: "What should I do for a patient who isn't responding
to antidepressants?"
  ↓ embed
Query Vector: [0.12, -0.34, 0.56, ...]
  ↓ search (cosine similarity)
Top-K Results: guidelines about treatment-resistant
depression
```

Keyword search would miss this because the query doesn't contain "treatment-resistant". **Semantic search** finds it because the *meaning* matches.

```
In [ ]: # — Semantic search with ChromaDB —————

queries = [
    "What should I do for a patient who isn't responding to antidepressants?",
    "How do I assess suicide risk?",
    "What therapy works best for panic attacks?",
```

```

    "Can exercise help with depression?",
]

print('=== Semantic Search Results ===\n')
for query in queries:
    results = collection.query(
        query_texts=[query],
        n_results=2 # top-2 most relevant
    )

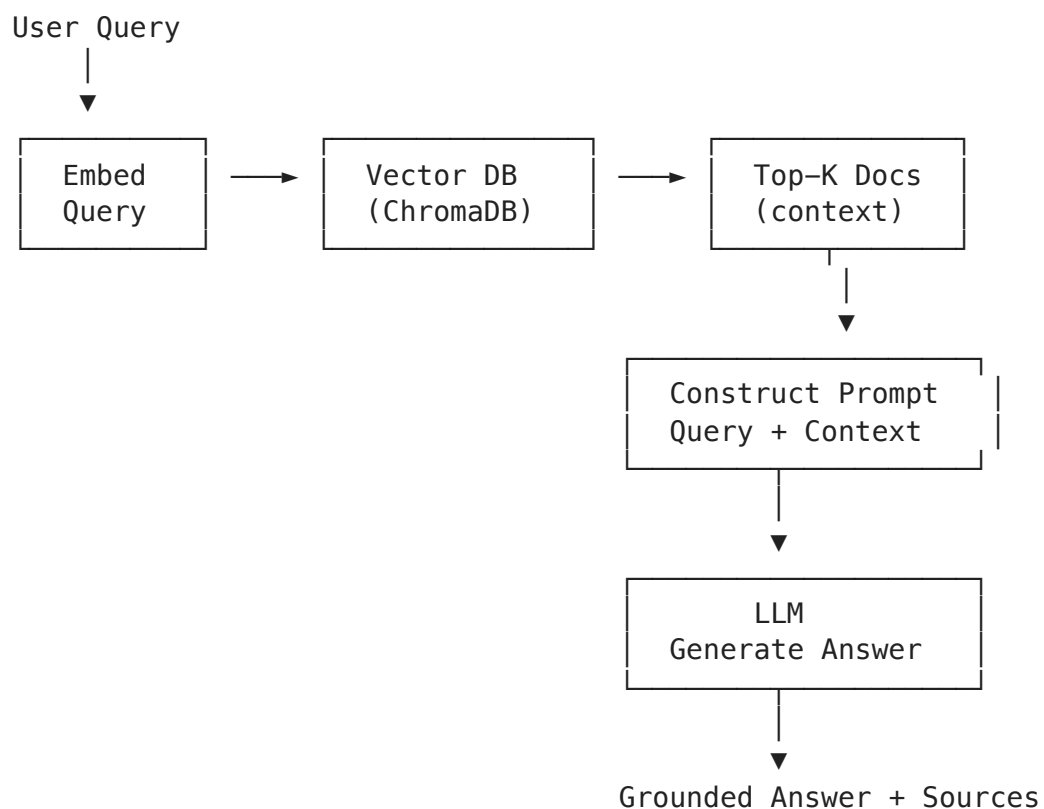
    print(f'Query: "{query}"')
    for i, (doc, dist) in enumerate(zip(results['documents'][0], results['distances'][0])):
        similarity = 1 - dist # ChromaDB returns distance, not similarity
        print(f' [{i+1}] (sim={similarity:.3f}) {doc[:100]}...')
    print()

print('💡 The search finds relevant guidelines by MEANING, not keyword match')
print('    "isn\'t responding to antidepressants" → "treatment-resistant depr

```

5. RAG Architecture — The Big Picture

RAG = Retrieval-Augmented Generation: combine search with generation.



Why RAG > Just Prompting?

Approach	Hallucination Risk	Knowledge Update	Context Window
Base LLM	High	Retrain (expensive)	Fixed at training

Approach	Hallucination Risk	Knowledge Update	Context Window
Fine-tuned LLM	Medium	Re-fine-tune	Fixed at training
RAG	Low	Update documents	Dynamic
RAG + Fine-tuned	Lowest	Update documents	Dynamic

Chunking Strategies

Documents must be split into chunks before embedding. The chunking strategy significantly impacts retrieval quality.

```
In [ ]: # — Chunking strategies —————

def chunk_fixed_size(text, chunk_size=200, overlap=50):
    """Split text into fixed-size character chunks with overlap."""
    chunks = []
    start = 0
    while start < len(text):
        end = start + chunk_size
        chunks.append(text[start:end])
        start = end - overlap # overlap with previous chunk
    return chunks

def chunk_by_sentence(text, max_chunk_size=300):
    """Split text into chunks at sentence boundaries."""
    import re
    sentences = re.split(r'(?<=[.!?])\s+', text)
    chunks, current_chunk = [], ''

    for sentence in sentences:
        if len(current_chunk) + len(sentence) > max_chunk_size and current_c
            chunks.append(current_chunk.strip())
            current_chunk = sentence
        else:
            current_chunk += ' ' + sentence
    if current_chunk.strip():
        chunks.append(current_chunk.strip())
    return chunks

# Demo with a longer clinical document
clinical_doc = (
    "Depression is a common mental health disorder affecting more than 280 m
    "It is characterised by persistent sadness, loss of interest, and inabil
    "The PHQ-9 is the gold standard screening tool for depression. A score c
    "10–14 moderate, 15–19 moderately severe, and 20–27 severe depression. "
    "First-line treatment includes psychotherapy (particularly CBT) and/or a
    "SSRIs such as sertraline and fluoxetine are preferred due to their favo
    "Treatment response should be assessed at 4–6 weeks. If inadequate respo
    "switching medication, or augmentation strategies."
)

fixed_chunks = chunk_fixed_size(clinical_doc, chunk_size=200, overlap=50)
sentence_chunks = chunk_by_sentence(clinical_doc, max_chunk_size=200)
```

```

print('=== Fixed-Size Chunking (200 chars, 50 overlap) ===')
for i, c in enumerate(fixed_chunks):
    print(f'  Chunk {i}: [{len(c)} chars] "{c[:60]}..."')

print(f'\n=== Sentence-Based Chunking (max 200 chars) ===')
for i, c in enumerate(sentence_chunks):
    print(f'  Chunk {i}: [{len(c)} chars] "{c[:60]}..."')

print(f'\n💡 Sentence-based chunking preserves semantic boundaries.')
print(f'  Overlap ensures context isn\'t lost at chunk edges.')

```

6. Building a Complete RAG Pipeline

Let's build the full pipeline **from scratch** (no LangChain) so you understand every component.

```

In [ ]: # — Complete RAG pipeline from scratch —————

class MentalHealthRAG:
    """End-to-end RAG pipeline for mental health clinical guidelines."""

    def __init__(self, embedding_model_name='all-MiniLM-L6-v2'):
        self.embedder = SentenceTransformer(embedding_model_name)
        self.client = chromadb.Client()
        self.collection = self.client.create_collection(
            name='rag_knowledge_base',
            metadata={'hnsw:space': 'cosine'})
        self.doc_count = 0

    def add_documents(self, documents, chunk_size=300, overlap=50):
        """Chunk, embed, and store documents."""
        all_chunks = []
        all_ids = []
        all_metadata = []

        for doc_idx, doc in enumerate(documents):
            chunks = chunk_by_sentence(doc, max_chunk_size=chunk_size)
            for chunk_idx, chunk in enumerate(chunks):
                all_chunks.append(chunk)
                all_ids.append(f'doc{doc_idx}_chunk{chunk_idx}')
                all_metadata.append({
                    'source_doc': doc_idx,
                    'chunk_idx': chunk_idx,
                    'char_count': len(chunk)
                })

        # Embed and store
        embeddings = self.embedder.encode(all_chunks).tolist()
        self.collection.add(
            documents=all_chunks,
            embeddings=embeddings,
            ids=all_ids,

```



```

        metadatas=all_metadata
    )
    self.doc_count += len(all_chunks)
    return len(all_chunks)

def retrieve(self, query, top_k=3):
    """Retrieve most relevant chunks for a query."""
    query_embedding = self.embedder.encode([query]).tolist()
    results = self.collection.query(
        query_embeddings=query_embedding,
        n_results=top_k
    )
    return results

def build_prompt(self, query, retrieved_docs):
    """Construct the LLM prompt with retrieved context."""
    context = '\n\n'.join([
        f'[Source {i+1}]: {doc}'
        for i, doc in enumerate(retrieved_docs['documents'][0])
    ])

    prompt = (
        f'You are a clinical mental health assistant. Answer the question\n'
        f'based ONLY on the provided clinical guidelines. If the guidelines\n'
        f'do not contain relevant information, say so.\n\n'
        f'### Clinical Guidelines:\n{context}\n\n'
        f'### Question:\n{query}\n\n'
        f'### Answer:'
    )
    return prompt

def query(self, question, top_k=3):
    """Full RAG pipeline: retrieve → build prompt → (generate)."""
    retrieved = self.retrieve(question, top_k=top_k)
    prompt = self.build_prompt(question, retrieved)

    # In production, you'd pass this prompt to an LLM:
    # response = llm.generate(prompt)
    return {
        'question': question,
        'retrieved_docs': retrieved['documents'][0],
        'distances': retrieved['distances'][0],
        'prompt': prompt
    }

print('RAG pipeline class defined.')
print('Components: embed → chunk → store → retrieve → prompt → generate')

```

```

In [ ]: # — Build and test the RAG pipeline —————

# Clinical guideline documents (longer form)
clinical_documents = [
    "Depression is a common mental health disorder affecting more than 280 m",
    "It is characterised by persistent sadness, loss of interest, and inabil",
    "The PHQ-9 is the gold standard screening tool for depression. A score c",
    "10–14 moderate, 15–19 moderately severe, and 20–27 severe depression."
]

```

```

"First-line treatment includes psychotherapy (particularly CBT) and/or a
"SSRIs such as sertraline and fluoxetine are preferred due to their favo

"Generalised Anxiety Disorder (GAD) is characterised by excessive, uncor
"Diagnosis requires symptoms persisting for at least 6 months with signi
"The GAD-7 questionnaire is used for screening. Scores of 5, 10, and 15
"First-line treatments include CBT and SSRI/SNRI medication. Buspirone m
"Benzodiazepines should only be prescribed short-term (2–4 weeks) due to

"Suicidal ideation requires immediate and systematic risk assessment. "
"The Columbia Suicide Severity Rating Scale (C-SSRS) is the recommended
"Risk factors include previous attempts, substance abuse, social isolati
"Protective factors include strong social support, therapeutic alliance,
"High-risk patients require crisis intervention, safety planning, and po
"All clinical staff should be trained in suicide risk assessment protoc

"Treatment-resistant depression (TRD) is defined as inadequate response
"Augmentation strategies include adding lithium, atypical antipsychotics
"Switching to a different class of antidepressant (e.g., SNRI, TCA, MAOI
"Esketamine nasal spray is approved for TRD in adults. Electroconvulsive
"Combination therapy (medication + psychotherapy) shows superior outcome
]

# Build the RAG system
rag = MentalHealthRAG()
n_chunks = rag.add_documents(clinical_documents)
print(f'Indexed {n_chunks} chunks from {len(clinical_documents)} documents')

# Test queries
test_queries = [
    "What screening tool should I use for depression?",
    "My patient isn't responding to antidepressants. What are my options?",
    "How do I assess if a patient is at risk of suicide?",
]

print('\n=== RAG Pipeline Results ===\n')
for query in test_queries:
    result = rag.query(query, top_k=2)
    print(f'Q: {result["question"]}')
    for i, (doc, dist) in enumerate(zip(result['retrieved_docs'], result['di
        sim = 1 - dist
        print(f' Retrieved [{i+1}] (sim={sim:.3f}): {doc[:100]}...')
    print()

# Show the full prompt for the last query
print('=== Full LLM Prompt (last query) ===\n')
print(result['prompt'])
print('\n💡 This prompt would be sent to an LLM (GPT-4, Phi-3, etc.) for ger
print(' The answer is GROUNDED in retrieved documents – not hallucinated.'

```

7. Advanced RAG Techniques

Basic RAG has limitations. Here's how to improve it:

Hybrid Search

Combine **semantic search** (meaning) with **keyword search** (exact terms). Critical when queries contain specific codes, drug names, or abbreviations.

Re-ranking

Retrieve more candidates (top-20), then re-rank with a more powerful model to get the true top-3.

RAG Evaluation

Metric	What It Measures	How
Context Relevance	Are retrieved docs relevant to the query?	Embedding similarity
Answer Faithfulness	Is the answer grounded in the context?	NLI / LLM-as-judge
Answer Relevance	Does the answer address the question?	Embedding similarity

RAG vs Fine-tuning vs Both

Approach	Best For	Limitations
RAG only	Factual Q&A, document search	Depends on retrieval quality
Fine-tuning only	Style/format changes, domain adaptation	Knowledge is static
RAG + Fine-tuning	Production systems needing both	More complex pipeline

```
In [ ]: # — Hybrid search: keyword + semantic —————
import re
from collections import Counter

def keyword_search(query, documents, top_k=3):
    """Simple TF-based keyword search for comparison."""
    query_terms = set(re.findall(r'\w+', query.lower()))
    scores = []
    for doc in documents:
        doc_terms = Counter(re.findall(r'\w+', doc.lower()))
        score = sum(doc_terms.get(term, 0) for term in query_terms)
        scores.append(score)
    ranked = sorted(enumerate(scores), key=lambda x: x[1], reverse=True)
    return ranked[:top_k]

def hybrid_search(query, rag_pipeline, documents, top_k=3, alpha=0.7):
    """Combine semantic (alpha) and keyword (1-alpha) search scores."""
    # Semantic scores
    semantic_results = rag_pipeline.retrieve(query, top_k=len(documents))
    semantic_scores = {}
```

```

    for doc, dist in zip(semantic_results['documents'][0], semantic_results[
        semantic_scores[doc[:50]] = 1 - dist # convert distance to similarity

# Keyword scores (normalised)
keyword_results = keyword_search(query, documents)
max_kw = max(score for _, score in keyword_results) if keyword_results else 0
keyword_scores = {documents[idx][:50]: score / max(max_kw, 1)
                  for idx, score in keyword_results}

# Combine
combined = {}
for key in set(list(semantic_scores.keys()) + list(keyword_scores.keys())):
    sem = semantic_scores.get(key, 0)
    kw = keyword_scores.get(key, 0)
    combined[key] = alpha * sem + (1 - alpha) * kw

ranked = sorted(combined.items(), key=lambda x: x[1], reverse=True)
return ranked[:top_k]

# Compare search methods
query = "PHQ-9 screening tool for depression"
print(f'Query: "{query}"\n')

print('--- Keyword Search ---')
kw_results = keyword_search(query, guidelines, top_k=2)
for idx, score in kw_results:
    print(f' [score={score}] {guidelines[idx][:80]}...')

print('\n--- Semantic Search ---')
sem_results = collection.query(query_texts=[query], n_results=2)
for doc, dist in zip(sem_results['documents'][0], sem_results['distances'][0]):
    print(f' [sim={1-dist:.3f}] {doc[:80]}...')

print('\n💡 Hybrid search combines both: semantic understanding + exact term matching')
print(' Essential when queries contain specific medical codes like "PHQ-9"')

```

```

In [ ]: # — RAG evaluation: measuring retrieval quality —————

# Define test cases: query → expected relevant guideline index
eval_cases = [
    {'query': 'How to screen for depression?', 'relevant': [0]},
    {'query': 'Treatment options for anxiety disorder', 'relevant': [3, 4]},
    {'query': 'Suicide risk assessment protocol', 'relevant': [6]},
    {'query': 'What to do when antidepressants don't work', 'relevant': [2]},
    {'query': 'Recommended therapy for panic attacks', 'relevant': [5]},
]

def evaluate_retrieval(collection, eval_cases, guidelines, k=3):
    """Compute retrieval precision and recall at k."""
    precisions, recalls = [], []

    for case in eval_cases:
        results = collection.query(query_texts=[case['query']], n_results=k)
        retrieved_docs = results['documents'][0]

        # Check which retrieved docs match the expected relevant ones

```

```

relevant_set = set(guidelines[i] for i in case['relevant'])
retrieved_set = set(retrieved_docs)

hits = len(relevant_set & retrieved_set)
precision = hits / k
recall = hits / len(case['relevant'])
precisions.append(precision)
recalls.append(recall)

return np.mean(precisions), np.mean(recalls)

for k in [1, 2, 3, 5]:
    prec, rec = evaluate_retrieval(collection, eval_cases, guidelines, k=k)
    print(f'k={k}: Precision@{k}={prec:.3f}, Recall@{k}={rec:.3f}')

print('\n💡 As k increases, recall improves (more chances to find the right
print('    but precision drops (more irrelevant docs in the results).')
print('    In RAG, k=3-5 is usually the sweet spot.')

```

Exercises

1. **Expand the knowledge base:** Add 5 more clinical guidelines (PTSD, OCD, eating disorders, substance abuse, child/adolescent mental health) and test retrieval quality.
2. **Chunk size experiment:** Re-index the documents with chunk sizes of 100, 200, 500, and 1000 characters. Measure Precision@3 and Recall@3 for each. What chunk size works best?
3. **Hybrid search implementation:** Build a complete hybrid search that combines ChromaDB semantic results with TF-IDF keyword matching. Test with queries containing specific medical terms ("PHQ-9", "sertraline", "C-SSRS").
4. **Manual RAG evaluation:** For 5 test questions, retrieve the top-3 documents, manually rate their relevance (1-5 scale), and compute the average relevance score.
5. **Different embedding models:** Compare retrieval quality between `all-MiniLM-L6-v2` (384d, fast) and `all-mpnet-base-v2` (768d, more accurate). Is the quality difference worth the speed cost?

Key Takeaways

1. **Embeddings** convert text to vectors where similarity = proximity in space
2. **Cosine similarity** is the standard metric for comparing embeddings
3. **Vector databases** (ChromaDB, FAISS, Pinecone) enable fast similarity search at scale using ANN algorithms
4. **RAG** = Retrieve relevant context + Generate answer grounded in that context

5. **Chunking strategy** significantly impacts RAG quality — sentence-based with overlap works well
6. **RAG reduces hallucination** and keeps knowledge updatable without model retraining
7. **Evaluate RAG** on retrieval relevance, answer faithfulness, and answer completeness

Next: [15 — Prompt Engineering & LLM Patterns](#) — designing effective prompts, structured output, agents, and more